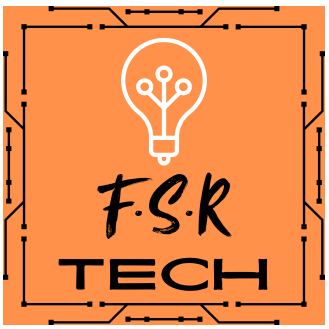


SIGSM

Sistemas Operativos

Nombre de la empresa: FSR TECH

Grupo: 3°MD

Unidad curricular: Sistemas Operativos

Turno: Vespertino

Nombres de los integrantes: Santiago Vazquez, Romina Pardo, Facundo Caraballo y José Motte

Docente: Garcia, Daniel

fecha de entrega: 27/07/2026

Escuela Superior de Comercio Villa Muñoz

Constitución 2484 esq. Rivadavia - Montevideo

Índice

1- Introducción.....3
1.1- Estudios de los diferentes roles de los usuarios del servidor.....3 y 4
1.2- Relevamiento y justificación del sistema operativo a utilizar.....4
1.3- Manual de instalación del sistema operativo en el servidor.....5
1.4- Primera versión del scripts de gestión de usuarios y grupos en el sistema operativo.....8,20

1- **Introducción:**

El presente documento tiene como finalidad describir los aspectos relacionados con el sistema y la administración del servidor que serán utilizados en el desarrollo del proyecto de egreso. El objetivo de este proyecto es desarrollar una aplicación web orientada al Hospital de Clínicas, que permitirá a los pacientes acceder a un portal mediante un código QR proporcionado por la institución. A través de dicho portal, los usuarios podrán consultar documentación médica y recomendaciones relacionadas con distintas categorías y servicios. Asimismo, el sistema permitirá completar encuestas de satisfacción accediendo con un código de pulsera, con el fin de recopilar opiniones y ayudar a mejorar los servicios ofrecidos.

Para garantizar el correcto funcionamiento de la plataforma se utilizará un servidor para almacenar usuarios, grupos, permisos y recursos del sistema. Por este motivo se realizará un estudio con los diferentes roles que hay en el servidor, la elección del sistema operativo a utilizar para la implementación de herramientas de administración que faciliten a los usuarios y grupos.

Se va a documentar el manual de instalación del sistema operativo en el servidor así como los scripts de gestión de usuarios.

1.1- **Estudios de los diferentes roles de los usuarios del servidor:**

1) **Administrador**: Es el usuario responsable de la coordinación general del sistema, teniendo el máximo nivel de acceso al mismo. Cuenta con todos los permisos necesarios para administrar, modificar, eliminar usuarios, crear documentos, encuestas y configuraciones generales de la plataforma.

2) **Auditor:** Se considera un usuario que su función es supervisar el sistema. Tiene acceso a los registros(logs) para verificar las modificaciones y operaciones realizadas por otros usuarios.

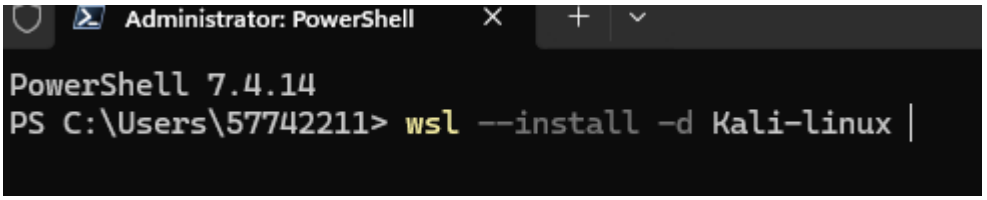
3) **Pacientes**:Son los usuarios que pueden acceder al sistema (portal) mediante un código QR. Este tiene que estar ubicado en lugares bien visibles de fácil acceso para que los usuarios puedan escanearlo correctamente. Así podrán obtener información e instrucciones, tratamientos, anuncios, etc, que le sea útil para resolver su problema de salud. También podrán acceder a encuestas de satisfacción mediante un código de pulsera que le dará el hospital. Para las personas que no están familiarizadas con la tecnología van a tener un apartado de ayuda para que entiendan el sistema.

1.2- **Relevamiento y justificación del Sistema Operativo a utilizar:**

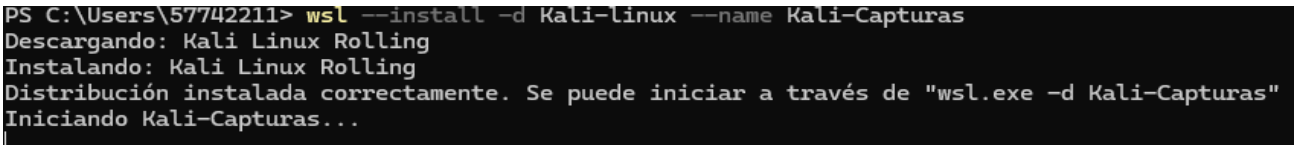
El sistema operativo que se eligió para el desarrollo del proyecto es Kali Linux porque es un sistema para el servidor. Aparte es de código abierto basado en Debian (sistema operativo de código abierto y software libre que funciona con el núcleo de Linux). Ofrece seguridad, estabilidad y herramientas para la administración de sistemas. Con este se permite la gestión de usuarios, grupos y permisos y los aspectos fundamentales para que funcione el servidor. Su entorno facilita la automatización de tareas a través de scripts en Bash, esto resulta muy útil para el mantenimiento del sistema. Por estos motivos se considera que Kali Linux es una alternativa adecuada para implementar el servidor y el desarrollo de la aplicación web propuesta.

1.3- **Manual de instalación del sistema operativo en el servidor:**

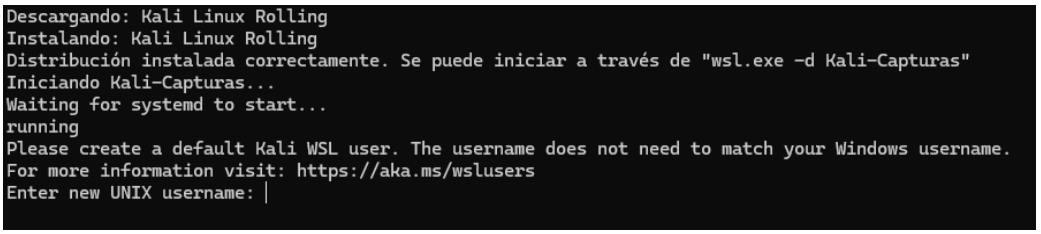
1- Escribir en la terminal de Windows (PowerShell):
wsl --install -d Kali-linux



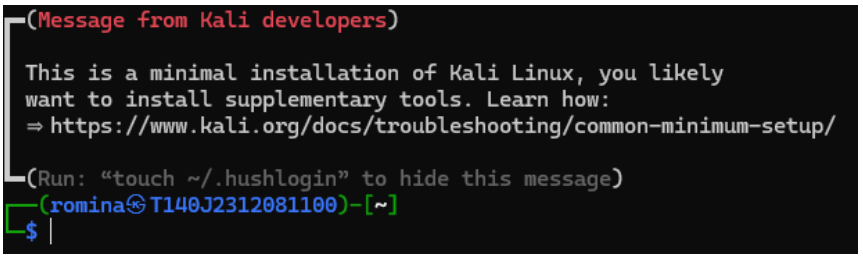
2- Se le tuvo que crear una copia de instalación de Kali linux con el comando wsl --install -d Kali-linux --name Kali-Capturas, cambiandole el nombre porque ya está instalado.



3- Ingresar nombre y contraseña de usuario:



4- El sistema esta pronto para usar:



5- **Instalación del servidor Apache en Kali Linux:**

```
┌──(Message from Kali developers)
│
│  This is a minimal installation of Kali Linux, you likely
│  want to install supplementary tools. Learn how:
│  ⇒ https://www.kali.org/docs/troubleshooting/common-minimum-setup/
│
└──(Run: "touch ~/.hushlogin" to hide this message)
┌──(romina㉿T140J2312081100)-[~]
└─$ sudo apt update
[sudo] password for romina:
Sorry, try again.
[sudo] password for romina:
Get:1 http://kali.download/kali kali-last-snapshot InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-last-snapshot/main amd64 Packages [21.2 MB]
Get:3 http://kali.download/kali kali-last-snapshot/main amd64 Contents (deb) [53.3 MB]
Get:4 http://kali.download/kali kali-last-snapshot/contrib amd64 Packages [103 kB]
Get:5 http://kali.download/kali kali-last-snapshot/contrib amd64 Contents (deb) [188 kB]
Get:6 http://kali.download/kali kali-last-snapshot/non-free amd64 Packages [175 kB]
Get:7 http://kali.download/kali kali-last-snapshot/non-free amd64 Contents (deb) [891 kB]
Get:8 http://kali.download/kali kali-last-snapshot/non-free-firmware amd64 Packages [15.8 kB]
Get:9 http://kali.download/kali kali-last-snapshot/non-free-firmware amd64 Contents (deb) [38.9 kB]
Fetched 76.0 MB in 1min 2s (1,221 kB/s)
206 packages can be upgraded. Run 'apt list --upgradable' to see them.

┌──(romina㉿T140J2312081100)-[~]
└─$ sudo apt full-upgrade -y
The following packages were automatically installed and are no longer required:
  librtmp1  libsodium23
Use 'sudo apt autoremove' to remove them.
```

```
┌──(romina㉿T140J2312081100)-[~]
└─$ sudo apt install apache2 -y
The following packages were automatically installed and are no longer required:
  librtmp1  libsodium23
Use 'sudo apt autoremove' to remove them.

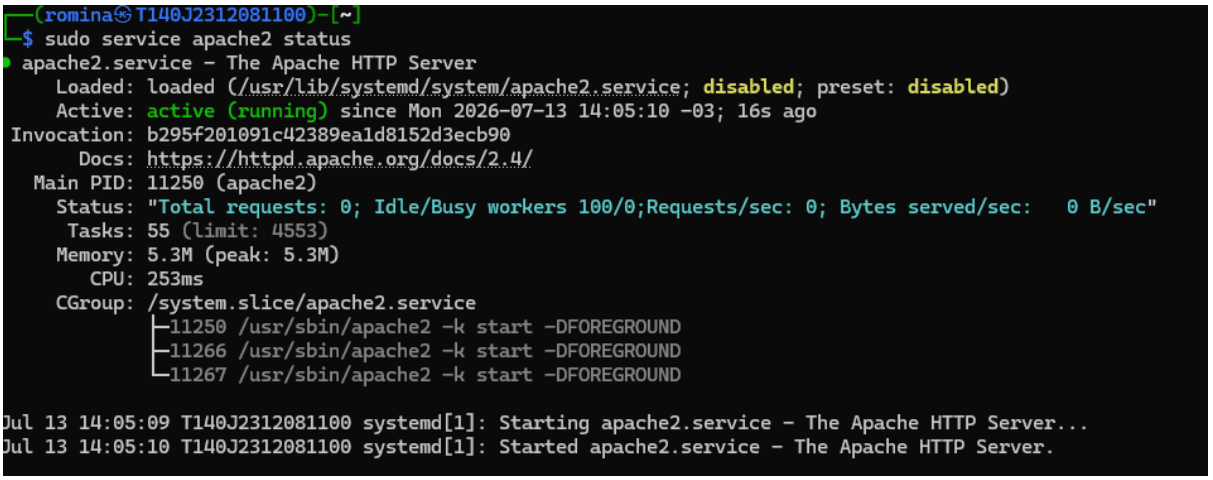
Installing:
  apache2

Installing dependencies:
  apache2-bin  apache2-data  apache2-utils  libapr1t64  libaprutil1-dbd-sqlite3  libaprutil1-ldap  libaprutil1t64  liblua5.4-0  ssl-cert

Suggested packages:
  apache2-doc  apache2-suexec-pristine  | apache2-suexec-custom  ufw  www-browser

Summary:
  Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 0
  Download size: 2,409 kB
  Space needed: 8,552 kB / 1,025 GB available

Get:1 http://http.kali.org/kali kali-last-snapshot/main amd64 libapr1t64 amd64 1.7.6-3+b1 [105 kB]
Get:2 http://http.kali.org/kali kali-last-snapshot/main amd64 libaprutil1t64 amd64 1.6.3-4+b1 [89.3 kB]
Get:3 http://http.kali.org/kali kali-last-snapshot/main amd64 libaprutil1-dbd-sqlite3 amd64 1.6.3-4+b1 [13.6 kB]
Get:4 http://http.kali.org/kali kali-last-snapshot/main amd64 libaprutil1-ldap amd64 1.6.3-4+b1 [11.6 kB]
Get:5 http://http.kali.org/kali kali-last-snapshot/main amd64 liblua5.4-0 amd64 5.4.8-1+b2 [148 kB]
Get:7 http://kali.download/kali kali-last-snapshot/main amd64 apache2-data all 2.4.68-1 [160 kB]
Get:8 http://kali.download/kali kali-last-snapshot/main amd64 apache2-utils amd64 2.4.68-1 [217 kB]
Get:9 http://kali.download/kali kali-last-snapshot/main amd64 apache2 amd64 2.4.68-1 [229 kB]
Get:10 http://kali.download/kali kali-last-snapshot/main amd64 ssl-cert all 1.1.3 [16.8 kB]
Get:6 http://elmirror.cl/kali kali-last-snapshot/main amd64 apache2-bin amd64 2.4.68-1 [1,419 kB]
Fetched 2,409 kB in 2s (1,039 kB/s)
Preconfiguring packages ...
Selecting previously unselected package libapr1t64:amd64.
(Reading database… 32451 files and directories currently installed.)
```


(romina㉿T140J2312081100)-[~]
$ sudo service apache2 status
apache2.service - The Apache HTTP Server
Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
Active: active (running) since Mon 2026-07-13 14:05:10 -03; 16s ago
Invocation: b295f201091c42389ea1d8152d3ecb90
Docs: https://httpd.apache.org/docs/2.4/
Main PID: 11250 (apache2)
Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
Tasks: 55 (limit: 4553)
Memory: 5.3M (peak: 5.3M)
CPU: 253ms
CGroup: /system.slice/apache2.service
├─11250 /usr/sbin/apache2 -k start -DFOREGROUND
├─11266 /usr/sbin/apache2 -k start -DFOREGROUND
└─11267 /usr/sbin/apache2 -k start -DFOREGROUND
Jul 13 14:05:09 T140J2312081100 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jul 13 14:05:10 T140J2312081100 systemd[1]: Started apache2.service - The Apache HTTP Server.

1.4- **Primera versión del scripts de gestión de usuarios y grupos en el sistema operativo:**

menú

```
#!/bin/bash
clear
verif_Logs="$(ls Logs.txt)"
clear
if [ -z "$verif_Logs" ]; then
> Logs.txt
fi
echo "Menú Gestión de Usuarios y datos de pacientes:"
echo ""
echo "1- Agregar Usuario"
echo "2- Borrar Usuario"
echo "3- Borrar grupo"
echo "4- Modificar Usuario"
echo "5- Lista de Usuarios"
echo "6- ingresar datos de paciente"
echo "7- descifrar datos de paciente"
echo ""
echo "0- Salir"
echo ""
read -p "Seleccione opción: " x
case $x in
1)
bash useradd
;;
2)
bash deluser
;;
3)
bash delgrup
;;
```

```
4)
bash moduser
;;
5)
bash lisuser;;
6)
bash ingresodatos;;
7)
bash decifrardatos
;;
0)
clear
echo "Cerrando menu.."
sleep 1
clear
exit
;;
*)
clear
echo "Opción incorrecta"
sleep 2
clear
sh menu
;;
esac
```

agregar usuario

```
#!/bin/bash
clear
while [ -z "$user" ]
do
read -p "Ingrese Nuevo Usuario: " user
sleep 2
clear
```

```
if [ -z "$user" ]; then
echo -n "El campo no puede estar vacío."
sleep 2
clear
fi
done
verif_user=$(cat /etc/passwd | cut -d":" -f1 | grep -w "$user")
if ! [ -z "$verif_user" ]; then
echo -n "El usuario $user ya existe"
sleep 2
clear
echo -n "Volviendo al Menu Principal.."
sleep 2
clear
sh menu
else
echo "Seleccione una opcion: "
echo "1- Crear un nuevo grupo para el usuario"
echo "2- Usar un grupo existente para el usuario"
read opc
case $opc in
1)
clear
while [[ -z "$grupoID" ]] || [[ "$grupoID" == "$verif_grupoID" ]]
do
echo -n "Ingrese ID de Grupo: "
read grupoID
verif_grupoID=$(cat /etc/group | cut -d":" -f3 | grep -w $grupoID)
if [ -z "$grupoID" ]; then
clear
echo -n "El campo no puede estar vacío."
sleep 2
```

```
clear
fi

if [[ "$grupoID" -eq "$verif_grupoID" ]]; then
echo -n "El ID de Grupo ingresado ya existe."
sleep 2
clear
fi
done
while [ -z "$grupoNombre" ]
do
echo -n "Ingrese Nombre de Grupo: "
read grupoNombre
if [ -z "$grupoNombre" ]; then
echo -n "El campo no puede estar vacío."
sleep 2
clear
fi
done
sudo groupadd -g "$grupoID" "$grupoNombre"
;;
2)
clear
echo -n "Ingrese Nombre de Grupo existente: "
read grupoNombre
verif_grupo=$(cat /etc/group | cut -d":" -f1 | grep -w $grupoNombre)
clear
if [ -z "$verif_grupo" ]; then
clear
echo -n "El grupo $grupoNombre no existe"
sleep 2
clear
```

```
echo -n "Volviendo al Menu Principal.."
sleep 2
clear
sh menu
fi
esac
echo "Creando usuario $user.."
sleep 3
clear
sudo useradd -d /home/$user -g $grupoNombre -p $user -m -s /bin/bash
$user
fecha_actual=$(date)
echo "(CREAR:USUARIO) (USUARIO) $user (FECHA) $fecha_actual"
>> Logs.txt
echo "Volviendo al Menu Principal.."
sleep 3
clear
sh menu
fi
modificar usuarios
#!/bin/bash
clear
while [ -z "$user" ]
do
read -p "Ingrese usuario a modificar: " user
if [ -z $user ]; then
clear
echo -n "El campo no puede estar vacío."
sleep 2
clear
fi
done
if [ $(cat /etc/passwd | cut -d":" -f1 | grep -w $user) = $user ]; then
clear
```

```
echo "Modificar Usuario:"
echo ""
echo "1- Cambiar Nombre"
echo "2- Cambiar Contraseña"
echo ""
echo "0- Volver atrás"
echo ""
read -p "Seleccione Opción: " x
case $x in
1)
clear
while [ -z "$newUser" ]
do
read -p "Ingrese Nuevo Nombre: " newUser
if [ -z $newUser ]; then
clear
echo -n "El campo no puede estar vacío."
sleep 2
clear
fi
done
sudo usermod -l $newUser $user
fecha_actual=$(date)
echo "(MODIFICAR:USUARIO:NOMBRE) (NUEVO USUARIO) $newUser
(ANTERIOR USUARIO) $user (FECHA) $fecha_actual" >> Logs.txt
clear
echo "Nombre modificado correctamente.."
sleep 3
clear
echo "Volviendo al menú principal.."
sleep 3
sh menu
$fecha_actual" >> Logs.txt
clear
echo "Volviendo al menú principal.."
sleep 3
sh menu
```

```
;;
0)
clear
echo "Volviendo al Menú Principal.."
sleep 3
sh menu
;;
*)
clear
echo "Opción incorrecta."
sleep 3
clea
sh moduser
;;
esac
else
clear
echo "El usuario ingresado no existe."
sleep 3
clear
echo "Volviendo al Menú.."
sleep 3
clear
sh moduser
fi
Listar usuarios
#!/bin/bash
clear
while [ -z "$user" ]
do
read -p "Ingrese usuario a modificar: " user
if [ -z $user ]; then
clear
echo -n "El campo no puede estar vacío."
sleep 2
clear
fi
```

```
done
if [ $(cat /etc/passwd | cut -d":" -f1 | grep -w $user) = $user ]; then
clear
echo "Modificar Usuario:"
echo ""
echo "1- Cambiar Nombre"
echo "2- Cambiar Contraseña"
echo ""
echo "0- Volver atrás"
echo ""
read -p "Seleccione Opción: " x
case $x in
1)
clear
while [ -z "$newUser" ]
do
read -p "Ingrese Nuevo Nombre: " newUser
if [ -z $newUser ]; then
clear
echo -n "El campo no puede estar vacío."
sleep 2
clear
fi
done
sudo usermod -l $newUser $user
fecha_actual=$(date)
echo "(MODIFICAR:USUARIO:NOMBRE) (NUEVO USUARIO) $newUser
(ANTERIOR USUARIO) $user (FECHA) $fecha_actual" >> Logs.txt
clear
echo "Nombre modificado correctamente.."
sleep 3
clear
echo "Volviendo al menú principal.."
sleep 3
sh menu.sh
$fecha_actual" >> Logs.txt
clear
```

```
echo "Volviendo al menú principal.."
sleep 3
sh menu.sh
;;
0)
clear
echo "Volviendo al Menú Principal.."
sleep 3
sh menu
;;
*)
clear
echo "Opción incorrecta."
sleep 3
clear
sh moduser
;;
esac
else
clear
echo "El usuario ingresado no existe."
sleep 3
clear
echo "Volviendo al Menú.."
sleep 3
clear
sh moduser
fi
```

Borrar usuarios

```
#!/bin/bash
clear
read -p 'Ingrese usuario a borrar: ' user
if [ $(cat /etc/passwd | cut -d":" -f1 | grep -w $user) = $user ]; then
sudo userdel -f $user
fecha_actual=$(date)
echo "(BORRAR:USUARIO) (USUARIO) $user (FECHA) $fecha_actual"
>> Logs.txt
clear
```

```
echo "El usuario $user ha sido borrado"
sleep 2
clear
echo -n "Volviendo al Menu Principal.."
sleep 2
clear
sh menu
else
clear
echo "El usuario no existe"
sleep 2
echo -n "Volviendo al Menu Principal.."
sleep 2
clear
sh menu
fi
```

borrarGrupo

```
#!/bin/bash
clear
echo "ingrese nombre de grupo a eliminar"
read gr
gr=$(echo "$gr" | xargs)
sudo groupdel "$gr"
if [ $? -eq 0 ]; then
sed -i "/^$gr$/d" grupos_creados.txt
echo "grupo eliminado correctamente"
else
echo "el grupo no existe"
fi
sleep 4
sh menu
```

Ingreso de datos de pacientes

```
#!/bin/bash
clear
read -p "ingrese nombre del paciente:" nom
read -p "ingrese apellido del paciente:" ape
read -p "ingrese la cedula:" ci
read -p "ingrese el tefono del paciente:" tel
echo "el paciente ¿posee problemas de salud (1-si/2-no)?"
read -p "seleccione una opción:" op
case $op in
```

```
1)
read -p "ingrese la enfermedad del paciente:" enf
read -s -p "ingrese una contraceña de acceso para los datos" clave
archivo="${ci}.txt"
echo "nombre: $nom" > "$archivo"
echo "apellido: $ape" >> "$archivo"
echo "cedula: $ci" >> "$archivo"
echo "telefono: $tel" >> "$archivo"
echo "enfermedad $enf" >> "$archivo"
openssl enc -aes-256-cbc -a -salt -pbkdf2 \
-pass pass:"$clave" \
-in "$archivo" \
-out "${ci}.enc"
rm "$archivo"
echo "se estan cifrando los dato...."
sleep 5
echo "los datos se han cifrado correctamente."
sleep 5
bash menu
;;
2)
read -s -p "ingrese una contraceña de acceso para los datos" clave
archivo="${ci}.txt"
echo "nombre: $nom" > "$archivo"
echo "apellido: $ape" >> "$archivo"
echo "cedula: $ci" >> "$archivo"
echo "telefono: $tel" >> "$archivo"
echo "enfermedad: no tiene" >> "$archivo"
openssl enc -aes-256-cbc -a -salt -pbkdf2 \
-pass pass:"$clave" \
-in "$archivo" \
-out "${ci}.enc"
rm "$archivo"
echo "se estan cifrando los dato...."
sleep 5
echo "los datos se han cifrado correctamente."
sleep 5
bash menu
;;
*)echo "error: bebe escribir 1 o 2"
```

;;
esac
sh menu

Descifrado de datos

```
#!/bin/bash
clear
read -p "ingrese la cedula del paciente: " ci
cat "${ci}.enc" | openssl enc -aes-256-cbc -d -a -pbkdf2
sleep 5
bash menu
```